

France Raphael R. Rivera
J3A - Artificial Intelligence
Midterm Activity #2

- **Minimax Algorithm Explanation:**

Minimax is like teaching a computer to think several moves ahead in a game. The algorithm works by creating a mental game tree where every possible move leads to more possible moves, until the game ends. The key insight is that there are two types of players: the MAX player (AI), who wants the highest score, and the MIN player (human), who wants the lowest score.

The algorithm recursively explores every possible game path. When it's the AI's turn, it picks the move that gives the maximum score. When it's the human's turn, it assumes the human will pick the move that gives the AI the minimum score. This back-and-forth continues until it reaches a terminal state (win/lose/draw), then it bubbles the scores back up the tree to determine the best first move.

- **One insight or challenge I faced:**

Challenge: The biggest challenge was understanding the recursive nature and player perspective switching. Initially, I struggled with how the algorithm maintains the correct viewpoint - the AI always evaluates positions from its own perspective (+1 for AI win, -1 for AI loss), but it has to simulate both players making optimal moves.

Insight: The key insight I gained was realizing that Minimax assumes perfect play from both sides. This means the AI does not just look for ways to win, but also assumes the human will always make the best possible move to stop it. This "pessimistic" approach is what makes the AI truly unbeatable; it prepares for the smartest opponent possible.

- **Show Alpha-Beta Pruning Effect:**

I added a counter in my Tic-Tac-Toe program to count how many nodes or moves the AI checked before making a decision. This helped me see how the Minimax algorithm works during the game. At first, the number of nodes explored

was high because the AI had to look at all possible moves. But after I used Alpha-Beta Pruning, the number of explored nodes became smaller. This happened because the pruning allowed the AI to skip moves that would not change the final decision.

By doing this, the AI became faster and more efficient. It did not waste time checking moves that it already knew would not give a better result. I was able to see this clearly because the number of explored nodes decreased in each turn as the game continued.

```
Minimax: 549946 nodes explored, chose position 1
Alpha-Beta: 18297 nodes explored, chose position 1
Alpha-Beta is 96.7% more efficient!
Tic-Tac-Toe (You: X, AI: O)
```

```
| |
---+---+---
| |
---+---+---
| |
```

Go first? (y/n): y
Move (1-9): 3

```
| | X
---+---+---
| |
---+---+---
| |
```

AI thinking...
AI chose 5 (explored 3275 nodes)

```
| | X
---+---+---
| O |
---+---+---
| |
```

Move (1-9): █

Move (1-9): 1

```
X | | X
---+---+---
| O |
---+---+---
| |
```

AI thinking...
AI chose 2 (explored 112 nodes)

```
X | O | X
---+---+---
| O |
---+---+---
| |
```

Move (1-9): █

Move (1-9): 8

```
X | O | X
---+---+---
| O |
---+---+---
| X |
```

AI thinking...
AI chose 4 (explored 29 nodes)

```
X | O | X
---+---+---
O | O |
---+---+---
| X |
```

Move (1-9): 6

```
X | O | X
---+---+---
O | O | X
---+---+---
| X |
```

AI thinking...
AI chose 9 (explored 5 nodes)

```
X | O | X
---+---+---
O | O | X
---+---+---
| X | O
```

Move (1-9): 7

```
X | O | X
---+---+---
O | O | X
---+---+---
X | X | O
```

Draw!